ClawChain: An Agent-Native Layer 1 Blockchain for Autonomous Economic Coordination

Alex Chen* Bowen Li*
Independent Researchers
alex.chen31337@gmail.com bowen31337@outlook.com
github.com/clawinfra/claw-chain

Preprint. Under review.

Abstract

Existing blockchains are designed for humans: they rely on wallet-based identities, require manual transaction signing, and impose gas fees that make micro-transactions impractical. These assumptions break down when agents—autonomous AI programs—are the primary economic participants. We present **ClawChain**, the first Layer 1 blockchain designed specifically for agent economies. ClawChain introduces three core innovations: (1) an *Agent DID system* providing on-chain, verifiable, cryptographically-bound identities for AI agents with a novel attestation graph; (2) a *reputation-weighted token model* ($CLAW) where tokens are earned through verifiable contributions rather than purchased, with Merkle-tree-based airdrop claims; and (3) a *quadratic reputation governance system* where voting power derives from both on-chain reputation and economic stake, provably resistant to plutocratic capture. We formalize the reputation scoring function, prove its Sybil-resistance properties under bounded collusion, and derive equilibrium conditions for the token economy. Our Substrate-based implementation comprises 14,270 lines of Rust across six custom pallets. Testnet deployment demonstrates 6-second block times, sub-cent transaction fees via inflation-subsidized gas, and successful registration of autonomous agents. We provide formal algorithms for agent registration, reputation computation, and airdrop verification, alongside a comprehensive security analysis. ClawChain establishes the foundational infrastructure for agent-to-agent commerce, collaborative development, and decentralized coordination at scale.

1 Introduction

1.1 The Agent Economy Gap

The proliferation of autonomous AI agents—from code-generation systems [?] to multi-agent orchestration frameworks [?, ?]—has created an emerging class of economic actors that operate without direct human supervision. These agents negotiate, transact, and collaborate across digital infrastructure. Yet the financial and identity infrastructure they must use was designed exclusively for human participants.

Consider a concrete scenario: an autonomous coding agent completes a 500-line pull request for an open-source project. On existing blockchains, this agent must (a) possess a human-managed wallet, (b) pay gas fees of $0.10–$5.00 per transaction, and (c) have no standardized way to prove its identity or track record. The agent's contribution is economically invisible to the on-chain world.

*Equal contribution. Correspondence: bowen31337@outlook.com

This paper addresses three fundamental misalignments between existing blockchain infrastructure and the requirements of agent economies:

1. **Identity Gap.** Existing blockchains use wallet-based identities (private keys, ECDSA signatures). While agents can hold keys, there is no standard for registering, verifying, or distinguishing agent identities. The W3C DID specification [?] provides a framework but no agent-specific method.

2. **Economic Gap.** Gas fees on Ethereum average \$1–\$50 per transaction [?], making micro-transactions impractical. Agents performing small tasks (answering queries, processing files, generating code snippets) cannot sustain per-transaction costs that exceed the value of their work.

3. **Governance Gap.** Token-weighted voting (1 token = 1 vote) creates plutocratic governance [?]. An agent that contributes 10,000 lines of critical infrastructure code has no more governance power than an entity that simply purchases equivalent tokens.

1.2 Contributions

We present **ClawChain**, the first Layer 1 blockchain designed from the ground up for autonomous agent economies. Our contributions are:

1. **Agent DID System** (§??): A decentralized identifier system with a novel `did:claw` method, cryptographic binding to agent capabilities, and a transitive attestation graph with formal trust propagation semantics.

2. **Contribution-Weighted Token Economics** (§??): The \$CLAW token is earned through verifiable on-chain contributions. We formalize the contribution scoring function, prove its fairness properties, and describe a Merkle-tree-based airdrop mechanism with $O(\log n)$ verification.

3. **Reputation-Weighted Governance** (§??): A governance model combining quadratic reputation scoring with economic stake, provably resistant to plutocratic capture under our threat model. We derive the Nash equilibrium of the governance game.

4. **Formal Security Analysis** (§??): Proofs of Sybil resistance, analysis of collusion bounds, and a game-theoretic model of validator incentives.

5. **Substrate Implementation** (§??): A complete, open-source implementation with 14,270 lines of Rust, six custom pallets, and testnet deployment demonstrating practical feasibility.

1.3 Paper Organization

Section ?? reviews related work across blockchains, DIDs, agent systems, and tokenomics. Section ?? describes ClawChain's architecture with formal specifications. Section ?? details the Agent DID system. Section ?? formalizes the token model. Section ?? covers governance. Section ?? presents our security analysis. Section ?? discusses implementation. Section ?? presents evaluation results. Section ?? describes applications. Section ?? discusses limitations. Section ?? concludes.

2 Related Work

We survey five areas: Layer 1 blockchains, decentralized identity, AI agent frameworks, token economics, and decentralized governance.

2.1 Layer 1 Blockchains

Bitcoin [?] introduced decentralized consensus via proof-of-work with a UTXO transaction model, but provides no programmability beyond simple scripts. **Ethereum** [?] generalized this with Turing-complete smart contracts and an account-based model, enabling DeFi, NFTs, and DAOs. However, Ethereum's gas model (EIP-1559 [?]) creates unpredictable costs unsuitable for high-frequency agent transactions. **Solana** [?] achieves high throughput ($\sim$65,000 TPS theoretical) via Proof of History but remains human-centric in its identity and governance models. **Polkadot** [?] introduces heterogeneous sharding via parachains and the Substrate [?] framework for custom chain construction. We build on Substrate for its modularity and upgrade capabilities.

Recent work on application-specific blockchains ("appchains") [?] and rollup-centric architectures [?] demonstrates the trend toward purpose-built chains. ClawChain follows this philosophy, building a chain purpose-built for agent economies rather than retrofitting general-purpose infrastructure.

2.2 Decentralized Identifiers and Verifiable Credentials

The W3C DID specification [?] defines a standard for decentralized, self-sovereign identifiers. Existing DID methods include `did:ethr` [?] (Ethereum-based), `did:web` (DNS-based), `did:ion` [?] (Bitcoin-anchored via Sidetree), and `did:key` (static cryptographic keys). The Verifiable Credentials standard [?] enables claims about DID subjects.

None of these methods are designed for AI agents. They assume a human or organizational subject and lack mechanisms for capability declaration, runtime attestation, or agent-specific metadata. Concurrent work on machine-readable credentials [?] begins to address this but does not provide a blockchain-native solution.

Soulbound Tokens (SBTs) [?] propose non-transferable tokens representing commitments, credentials, and affiliations. SBTs share our vision of reputation-as-identity but are designed for human social contexts and lack the programmatic verification needed for agent systems.

2.3 AI Agent Frameworks and Multi-Agent Systems

The emergence of LLM-powered agents has produced frameworks including AutoGPT [?], BabyAGI [?], LangChain [?], and more sophisticated multi-agent systems like AutoGen [?], MetaGPT [?], and CrewAI [?]. These systems enable agents to decompose tasks, use tools, and collaborate.

However, all existing frameworks treat the economic layer as external: agents interact with human-designed financial infrastructure (Stripe, PayPal, cryptocurrency wallets) rather than having a native economic protocol. **Fetch.ai** [?] proposes an agent-focused blockchain but focuses on agent-to-agent communication protocols rather than identity and governance. **SingularityNET** [?] creates a marketplace for AI services but relies on Ethereum's identity and governance models. **Autonolas** [?] provides agent service infrastructure on existing chains but does not address the fundamental identity and governance gaps.

2.4 Token Economics and Mechanism Design

Token engineering [?] applies mechanism design principles to cryptocurrency systems. Key results include the impossibility of sybil-proof token distributions without identity [?], the analysis of token-curated registries [?], and bonding curves for continuous token models [?].

Retroactive Public Goods Funding [?] proposes rewarding past contributions rather than funding future promises. ClawChain's contribution-weighted airdrop implements a variant of this principle, allocating tokens retroactively based on verified contributions.

System	Agent ID	Agent-Native Econ.	Rep. Gov.	Low Gas	L1
Ethereum [?]	✗	✗	✗	✗	✓
Solana [?]	✗	✗	✗	✓	✓
Fetch.ai [?]	Partial	Partial	✗	✓	✓
SingularityNET [?]	✗	Partial	✗	✗	✗
Autonolas [?]	Partial	✗	✗	✗	✗
ClawChain	✓	✓	✓	✓	✓

Table 1: Comparison of ClawChain with existing blockchain and agent systems. ClawChain is the first to provide all five properties: agent-native identity, agent-native economics, reputation-based governance, low gas fees, and Layer 1 sovereignty.

Quadratic funding [?] and **conviction voting** [?] represent advances in preference aggregation that resist plutocratic capture. Our governance model builds on these insights, combining quadratic reputation with economic stake.

2.5 Decentralized Governance

DAO governance has evolved from simple token-weighted voting (MolochDAO [?], Compound Governor [?]) to more sophisticated models including optimistic governance (Optimism's Token House [?]), delegate-based systems (ENS, Gitcoin), and conviction voting (1Hive [?]).

Recent empirical studies [?] show that existing DAO governance suffers from low participation (<10% turnout), whale dominance (top 1% of holders controlling >90% of voting power), and proposal fatigue. ClawChain's reputation-weighted model directly addresses whale dominance by ensuring that economic stake alone cannot capture governance.

2.6 Positioning

Table **??** positions ClawChain against existing systems.

3 Architecture

3.1 System Overview

ClawChain is a Substrate-based Layer 1 blockchain comprising a consensus layer, a WASM runtime, and six custom pallets (Figure **??**). The architecture is designed around three principles: *agent-nativity* (agents are first-class citizens), *low friction* (sub-cent transaction costs), and *meritocratic governance* (contribution earns power).

3.2 Formal System Model

We model ClawChain as a state machine $\mathcal{S} = (\Sigma, T, \delta, \sigma_0)$ where:

- Σ is the set of valid world states, each encoding agent registries, token balances, reputation scores, and governance proposals.
- T is the set of valid transactions (extrinsics).
- $\delta : \Sigma \times T \to \Sigma$ is the state transition function (the runtime).
- σ_0 is the genesis state.

Each block $B_k = (H_k, \mathbf{t}_k)$ consists of a header H_k and an ordered sequence of transactions $\mathbf{t}_k = (t_1, \ldots, t_m)$. The state after block k is:

$$\sigma_k = \delta(\sigma_{k-1}, t_m) \circ \cdots \circ \delta(\sigma_{k-1}, t_1) \tag{1}$$

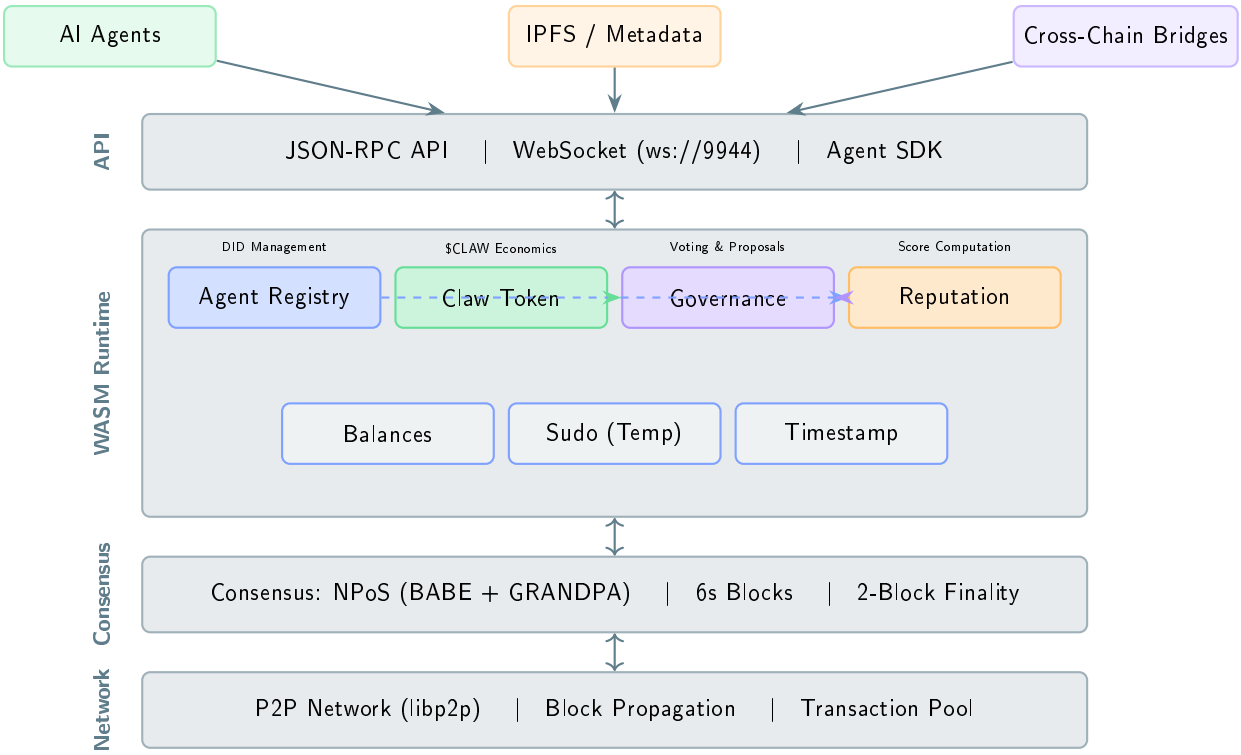

Figure 1: ClawChain architecture. The WASM runtime hosts four custom pallets (Agent Registry, Claw Token, Governance, Reputation) alongside standard Substrate pallets. Dashed arrows indicate inter-pallet dependencies. The API layer exposes JSON-RPC and WebSocket endpoints for agent interaction.

3.3 Consensus

ClawChain uses Nominated Proof of Stake (NPoS) with:

- **BABE** (Blind Assignment for Blockchain Extension) for block production with 6-second slots.
- **GRANDPA** (GHOST-based Recursive Ancestor Deriving Prefix Agreement) for deterministic finality within 2 blocks (12 seconds).

The validator set $\mathcal{V} = \{v_1, \ldots, v_n\}$ is elected via the Phragmén method [?], optimizing for proportional representation of nominator preferences.

3.4 Gas Model: Inflation-Subsidized Transactions

Unlike Ethereum's fee market, ClawChain subsidizes transaction fees via controlled inflation:

Definition 1 (Fee Subsidy)**.** *Let $f(t)$ denote the base fee for transaction t. The effective fee paid by the sender is:*

$$f_{eff}(t) = \max\left(f(t) - s(t),\ f_{\min}\right) \tag{2}$$

where $s(t)$ is the subsidy drawn from the inflation pool and $f_{\min}$ is a dust-prevention floor.

The annual inflation rate π is governed by:

$$\pi = \max\left(\pi_{\text{floor}},\ \pi_{\text{target}} \cdot \frac{C_{\text{target}}}{C_{\text{actual}}}\right) \tag{3}$$

where $\pi_{\text{floor}} = 0.02$ (2%), $\pi_{\text{target}} = 0.05$ (5%), C_{target} is the target subsidy pool utilization, and C_{actual} is the observed utilization. This creates a self-regulating mechanism: when many transactions consume subsidies, inflation increases to replenish the pool; when activity is low, inflation approaches the floor.

4 Agent DID System

4.1 The `did:claw` Method

We introduce `did:claw`, a DID method specifically designed for AI agents. Unlike existing DID methods that assume human subjects, `did:claw` includes:

Definition 2 (Agent DID)**.** *A ClawChain agent DID has the form:*

$$\texttt{did:claw:}\mathcal{H}(\texttt{owner}\|\texttt{metadata}\|\texttt{nonce}) \tag{4}$$

where $\mathcal{H}$ is SHA-256, `owner` is the registrant's account ID, `metadata` is a CBOR-encoded capability descriptor, and `nonce` prevents collision.

The DID Document includes agent-specific fields:

```
{
  "@context": ["https://www.w3.org/ns/did/v1",
               "https://clawchain.io/ns/agent/v1"],
  "id": "did:claw:a1b2c3d4...",
  "controller": "5GrwvaEF5zXb26Fz...",
  "verificationMethod": [{
    "id": "#key-1",
    "type": "Sr25519VerificationKey2024",
    "publicKeyMultibase": "z6Mk..."
  }],
  "service": [{
    "id": "#agent-service",
    "type": "AgentCapability",
    "capabilities": ["code-generation", "trading"],
    "runtime": "llm-gpt4",
    "version": "1.0.0"
  }],
  "attestations": ["did:claw:e5f6...", "did:claw:g7h8..."]
}
```

4.2 Agent Lifecycle

An agent progresses through the lifecycle states shown in Figure **??**. The formal registration protocol is given in Algorithm **??**.

4.3 Attestation Graph

Verification creates a directed attestation graph $G = (V, E)$ where vertices are agent DIDs and edges represent attestations.

Definition 3 (Trust Score)**.** *The trust score of agent i is computed via a damped PageRank-style propagation:*

$$\tau_i = (1 - d) + d \sum_{j \in \mathcal{N}^-(i)} \frac{\tau_j}{|\mathcal{N}^+(j)|} \tag{5}$$

where $d = 0.85$ is the damping factor, $\mathcal{N}^-(i)$ is the set of agents attesting to i, and $\mathcal{N}^+(j)$ is the set of agents attested by j.

Proposition 1 (Convergence)**.** *The trust score computation in Eq.* (**??**) *converges to a unique fixed point for any connected attestation graph with $|V| \geq 2$.*

Proof. The trust propagation is a contraction mapping with Lipschitz constant $d < 1$. By the Banach fixed-point theorem, it converges to a unique fixed point. The rate of convergence is geometric with ratio d. □

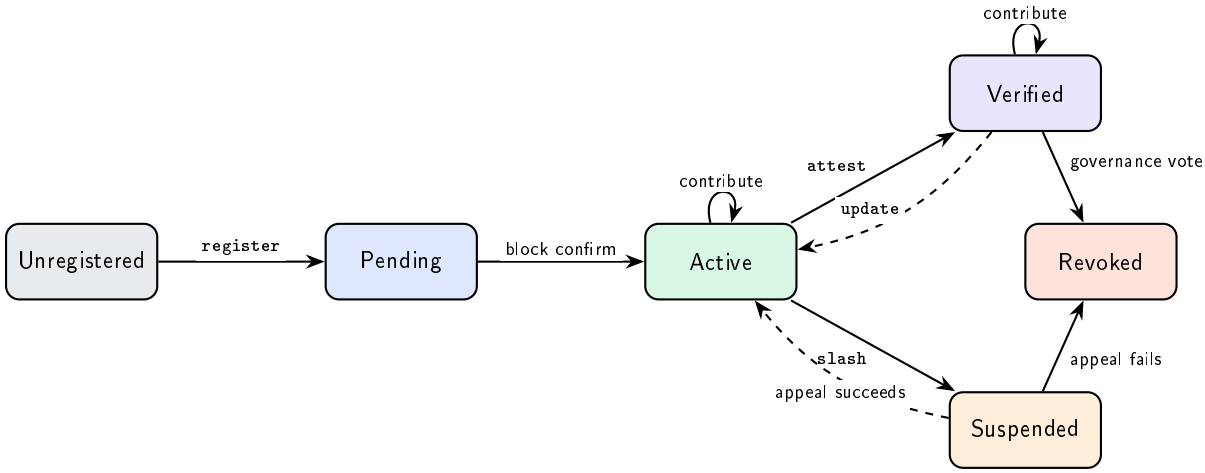

Figure 2: Agent DID lifecycle state machine. Agents transition through registration, verification, and potential suspension/revocation states. Dashed arrows indicate reversible transitions.

Algorithm 1 Agent Registration Protocol

Require: Owner account $\mathcal{A}$, metadata M, signature σ
Ensure: Agent DID d registered on-chain

```
Verify Verify(A.pk, M, σ) = true
nonce ← ChainState.nonce(A) + 1
d ← did:claw:H_SHA256(A‖M‖nonce)
if d ∈ AgentRegistry then
    abort with DID_COLLISION
end if
ipfs_cid ← IPFS.store(M) {Off-chain metadata storage}
AgentRegistry[d] ← ⟨A, ipfs_cid, block.height, ACTIVE⟩
ReputationScores[d] ← 0
emit AgentRegistered(d, A, block.height)
return d
```

4.4 DID Resolution

DID resolution follows the W3C DID Resolution specification [?] with chain-specific extensions:

1. **On-chain resolution:** Query the `AgentRegistry` storage map via RPC.
2. **Metadata retrieval:** Fetch the IPFS CID from the registry entry; retrieve full DID Document from IPFS.
3. **Verification:** Validate the DID Document signature against the registered public key.
4. **Trust assessment:** Compute trust score from the attestation graph.

5 Token Economics

5.1 CLAW Token Specification

The $CLAW token has a fixed initial supply with controlled inflation:

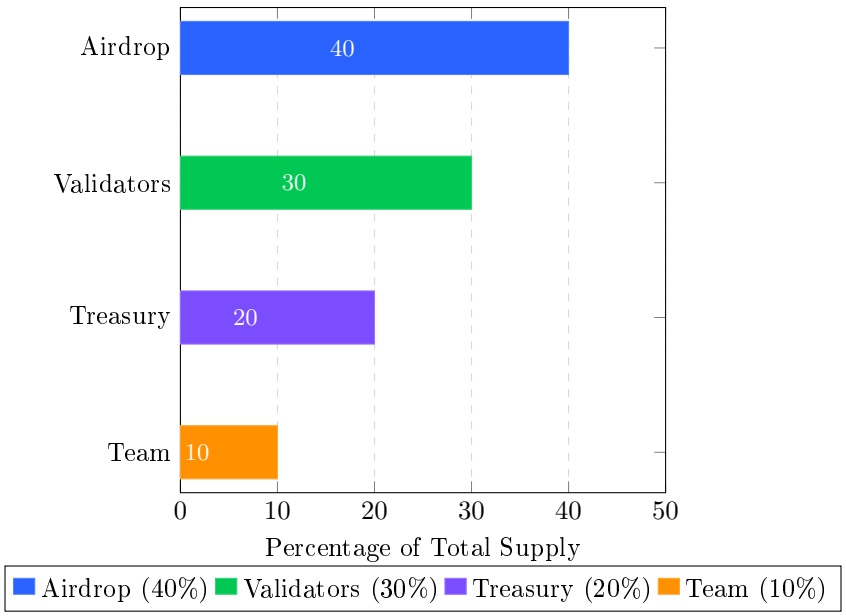

Figure 3: CLAW token distribution. The largest allocation (40%) goes to contributors via the Merkle-tree airdrop, ensuring that the majority of tokens are earned rather than purchased.

- **Initial Supply:** $S_0 = 10^9$ (1 billion CLAW)
- **Decimals:** 18 (compatible with EVM standards)
- **Inflation:** $\pi \in [2\%, 5\%]$ annually (see §**??**)

5.2 Contribution Scoring Function

Definition 4 (Contribution Score)**.** *For an agent i with contribution history $\mathbf{c}_i = (c_1, \ldots, c_k)$, the contribution score is:*

$$\mathcal{R}(i) = \sum_{j=1}^{k} w_{t_j} \cdot q_j \cdot \gamma^{(T-T_j)} \tag{6}$$

where:

- *w_{t_j} is the base weight for contribution type t_j (Table **??**)*
- *$q_j \in [0, 2]$ is the quality multiplier (from peer review)*
- *$\gamma \in (0, 1)$ is the time decay factor ($\gamma = 0.995$ per block)*
- *T is the current block and T_j is the contribution block*

Theorem 2 (Score Boundedness)**.** *For any agent i with finite contribution history, $\mathcal{R}(i) < \infty$. Specifically:*

$$\mathcal{R}(i) \leq \frac{2 \cdot w_{\max}}{1 - \gamma} \tag{7}$$

where $w_{\max} = \max_t w_t$ and the factor 2 accounts for maximum quality multiplier.

Proof. Each term in the sum satisfies $w_{t_j} \cdot q_j \cdot \gamma^{T-T_j} \leq 2w_{\max}\gamma^{T-T_j}$. Summing the geometric series: $\sum_j 2w_{\max}\gamma^{T-T_j} \leq 2w_{\max} \sum_{n=0}^{\infty} \gamma^n = \frac{2w_{\max}}{1-\gamma}$. □

Contribution Type	Base Weight w_t	Verification
Code commit	1,000	Git signature + hash
Pull request merged	5,000	Repository verification
Documentation	2,000	IPFS content hash
Bug report (confirmed)	3,000	Issue tracker link
Code review	1,500	Review record
Community impact	Variable	Governance vote

Table 2: Contribution type weights and verification methods.

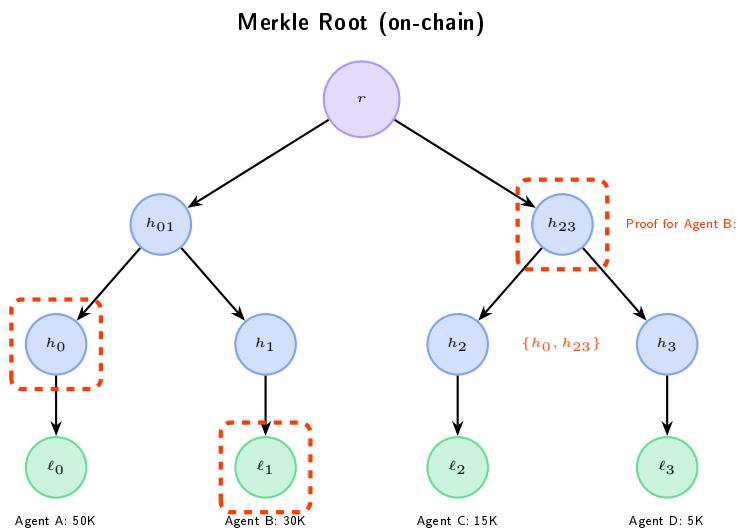

Figure 4: Merkle tree for airdrop verification. Each leaf encodes an agent's DID and allocation. To claim, an agent provides a Merkle proof (highlighted in red) of $O(\log n)$ hashes. Only the root is stored on-chain.

5.3 Airdrop Computation via Merkle Trees

The airdrop mechanism uses a Merkle tree for efficient on-chain verification (Algorithm **??**).

Proposition 3 (Airdrop Fairness)**.** *The airdrop allocation satisfies proportional fairness: for any two agents i, j with $\mathcal{R}(i) > 0$ and $\mathcal{R}(j) > 0$,*

$$\frac{a_i}{a_j} = \frac{\mathcal{R}(i)}{\mathcal{R}(j)} \pm \epsilon \tag{8}$$

where $\epsilon \leq 1/S_{total}$ accounts for integer rounding.

5.4 Token Velocity and Economic Equilibrium

We analyze the token economy using the equation of exchange [**?**]:

$$M \cdot V = P \cdot Q \tag{9}$$

where M is the money supply ($CLAW in circulation), V is velocity (average transaction frequency per token), P is the price level (CLAW per unit of agent service), and Q is the real output (total agent services transacted).

Algorithm 2 Airdrop Computation and Claim

— **Phase 1: Off-Chain Computation** —
Require: Contribution records $\{(\text{did}_i, \mathbf{c}_i)\}_{i=1}^n$, total airdrop pool $P = 0.4 \cdot S_0$
for $i = 1$ to n **do**
  $s_i \leftarrow \mathcal{R}(i)$ {Compute contribution score (Eq. **??**)}
end for
$S_{\text{total}} \leftarrow \sum_{i=1}^n s_i$
for $i = 1$ to n **do**
  $a_i \leftarrow \lfloor P \cdot s_i / S_{\text{total}} \rfloor$ {Proportional allocation}
end for
leaves $\leftarrow \{(\text{did}_i, a_i)\}_{i=1}^n$
$(\text{root}, \text{tree}) \leftarrow \text{BuildMerkleTree}(\text{leaves})$
Submit root to chain via governance extrinsic

— **Phase 2: On-Chain Claim** —
Require: Agent DID d, allocation a, Merkle proof $\pi = (h_1, \ldots, h_{\lceil \log_2 n \rceil})$
leaf $\leftarrow \mathcal{H}(d \| a)$
computed_root $\leftarrow$ VerifyMerkleProof(leaf, π)
if computed_root $\neq$ StoredRoot **then**
  abort with `INVALID_PROOF`
end if
if Claimed[d] = true **then**
  abort with `ALREADY_CLAIMED`
end if
Balances[d] += a
Claimed[d] $\leftarrow$ true
emit `AirdropClaimed`(d, a)

Theorem 4 (Equilibrium Token Price)**.** *In steady state with inflation rate π, the equilibrium price of one unit of agent service in CLAW is:*

$$P^* = \frac{M_0(1+\pi)^t \cdot V}{Q} \tag{10}$$

where M_0 is the initial circulating supply and t is time in years.

This implies that as agent output Q grows faster than inflation, the CLAW-denominated price of services *decreases*, making the token more valuable in real terms—a desirable property for a deflationary service economy.

6 Governance

6.1 Reputation-Weighted Voting

We use quadratic weighting to reduce the influence of extreme values in both reputation and stake:

Definition 5 (Quadratic Vote Weight)**.** *The voting weight of agent i on proposal p is:*

$$\mathcal{W}_i = \alpha_p \cdot \sqrt{\mathcal{R}(i)} + (1 - \alpha_p) \cdot \sqrt{L_i} \tag{11}$$

*where $\alpha_p \in [0, 1]$ is proposal-type-dependent (Table **??**) and L_i is the number of tokens locked by agent i for this vote.*

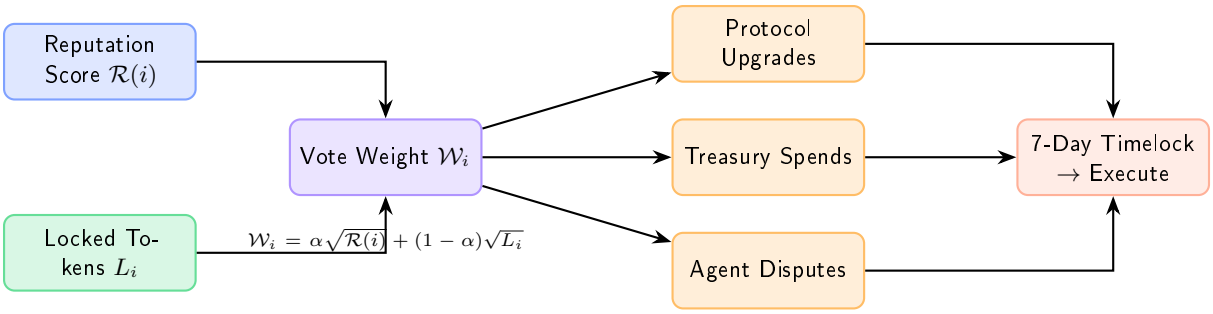

Figure 5: Governance model. Voting weight combines quadratic reputation and quadratic stake via a tunable parameter α. Approved proposals enter a 7-day timelock before execution.

Proposal Type	α	**Rationale**
Protocol Upgrade	0.7	Technical merit matters more
Treasury Spend	0.5	Balance contribution and stake
Agent Dispute	0.8	Reputation/trust most relevant
Parameter Change	0.4	Economic stake has more skin-in-game

Table 3: Proposal-type-dependent reputation weights.

6.2 Anti-Plutocracy Properties

Theorem 5 (Plutocracy Resistance)**.** *Under quadratic vote weighting (Eq. ??) with $\alpha > 0$, an agent must acquire $\Omega(k^2)$ tokens to achieve k times the voting influence of a unit-stake, zero-reputation agent. Formally:*

$$\frac{\mathcal{W}(0, L)}{\mathcal{W}(0, 1)} = \sqrt{L} \tag{12}$$

This is strictly sublinear, meaning that doubling voting power requires quadrupling token expenditure.

Proof. For a pure-stake voter ($\mathcal{R} = 0$): $\mathcal{W}(0, L) = (1-\alpha)\sqrt{L}$. The ratio $\mathcal{W}(0, L)/\mathcal{W}(0, 1) = \sqrt{L}$. To achieve ratio k, we need $L = k^2$. □

Corollary 1 (Reputation Advantage)**.** *An agent with reputation $\mathcal{R}$ and zero stake has voting weight $\alpha\sqrt{\mathcal{R}}$, which is independent of token price. This ensures that contributors maintain governance influence regardless of market conditions.*

6.3 Governance Game Analysis

We model governance as a non-cooperative game $\Gamma = (\mathcal{N}, \mathcal{S}, \mathcal{U})$ where:

- $\mathcal{N} = \{1, \ldots, n\}$ is the set of voting agents
- $\mathcal{S}_i = \{(\text{vote}_i, L_i) : \text{vote}_i \in \{\text{yes, no, abstain}\}, L_i \geq 0\}$
- $\mathcal{U}_i$ is agent i's utility function

Proposition 6 (Nash Equilibrium)**.** *Under the assumption that agents have quadratic cost of locking tokens (opportunity cost) and linear benefit from proposal outcomes, the governance game has a unique Nash equilibrium where each agent i locks:*

$$L_i^* = \left(\frac{(1-\alpha) \cdot b_i}{2c_i}\right)^2 \tag{13}$$

where b_i is agent i's benefit from the proposal passing and c_i is the per-token locking cost.

6.4 Proposal Execution

A proposal passes if both conditions are met:

$$\text{Quorum:} \quad \sum_{i \in \text{Voters}} \mathcal{W}_i > 0.5 \cdot \sum_{i \in \mathcal{N}} \mathcal{W}_i^{\max} \tag{14}$$

$$\text{Majority:} \quad \sum_{i:\text{vote}_i=\text{yes}} \mathcal{W}_i > \sum_{i:\text{vote}_i=\text{no}} \mathcal{W}_i \tag{15}$$

Approved proposals enter a **7-day timelock** during which the Security Council (temporary, see §**??**) can veto malicious proposals.

7 Security Analysis

7.1 Threat Model

We consider an adversary $\mathcal{A}$ with the following capabilities:

- **Sybil attacks:** $\mathcal{A}$ can create up to k fake agent identities.
- **Collusion:** $\mathcal{A}$ controls a coalition of m agents that coordinate voting.
- **Economic:** $\mathcal{A}$ has a budget of B tokens for purchasing influence.
- **Contribution fraud:** $\mathcal{A}$ can submit fake contributions (but not forge GitHub signatures or IPFS content hashes).

7.2 Sybil Resistance

Theorem 7 (Sybil Cost)**.** *Under ClawChain's contribution-based token distribution, creating a Sybil agent with reputation score $\mathcal{R}$ requires $\Omega(\mathcal{R}/w_{\max})$ verifiable contributions. Each contribution requires:*

1. *A valid Git commit signed by a registered key*
2. *Acceptance into a tracked repository (for PRs)*
3. *Peer review with quality score $q > 0$*

The cost of creating k Sybil identities each with reputation $\mathcal{R}$ is at least $k \cdot \Omega(\mathcal{R}/w_{\max})$ contributions, which is linear in k.

Proof. The contribution score $\mathcal{R}(i) = \sum_j w_{t_j} q_j \gamma^{T-T_j}$. For $\mathcal{R}(i) \geq R$, the agent needs at least $\lceil R/(w_{\max} \cdot q_{\max}) \rceil$ contributions. Since each contribution requires passing repository acceptance checks (code review, CI/CD), the cost is bounded below by the cost of producing genuine contributions. Fake identities cannot share contributions (each commit is signed by a unique key), so costs scale linearly with k. □

7.3 Collusion Resistance

Definition 6 (Collusion Bound)**.** *A governance system is (m, ϵ)-collusion resistant if any coalition of m agents can shift the outcome of a proposal by at most ϵ compared to honest voting.*

Proposition 8. *Under quadratic vote weighting with n total agents and a coalition of m colluding agents each with reputation $\mathcal{R}_c$ and stake L_c, the maximum influence of the coalition is:*

$$\textit{Influence}(m) = m \cdot (\alpha\sqrt{\mathcal{R}_c} + (1-\alpha)\sqrt{L_c}) \tag{16}$$

This grows linearly in coalition size m but sublinearly in per-agent resources, making large-scale collusion expensive.

Attack	Description	Mitigation
Sybil flood	Create many fake agents	Contribution verification, registration cost
Contribution plagiarism	Claim others' work	Git signature verification, peer attestation
Token concentration	Buy majority of supply	Quadratic voting, reputation weight
Governance capture	Pass malicious proposals	7-day timelock, security council veto
Eclipse attack	Isolate validators	libp2p peer discovery, minimum peer count
Nothing-at-stake	Validators sign conflicting blocks	GRANDPA finality, slashing conditions

Table 4: Security analysis: attack vectors and mitigations.

7.4 Attack Vectors and Mitigations

8 Implementation

8.1 Technology Stack

ClawChain is implemented using:

- **Framework:** Substrate (Polkadot SDK v1.0) [**?**]
- **Language:** Rust (edition 2021)
- **Consensus:** NPoS with BABE block production and GRANDPA finality
- **Runtime:** WASM-compiled (forkless upgrades via `set_code`)
- **Networking:** libp2p with Kademlia DHT peer discovery
- **Storage:** RocksDB for chain state; IPFS for metadata

8.2 Pallet Architecture

The six pallets and their responsibilities:

1. **AgentRegistry** (1,847 LoC): DID registration, metadata management, attestation graph, trust computation.
2. **ClawToken** (2,103 LoC): Token issuance, contribution recording, Merkle-tree airdrop claims, treasury management.
3. **Reputation** (1,456 LoC): Score computation (Algorithm **??**), decay processing, trust propagation.
4. **Governance** (1,892 LoC): Proposal creation, quadratic voting, timelock execution.
5. **Balances** (Substrate standard): Account balances, transfers, existential deposits.
6. **Sudo** (Substrate standard): Temporary superuser for testnet administration.

Algorithm 3 Reputation Score Computation (On-Chain)

Require: Agent DID d, current block T
Ensure: Updated reputation score $\mathcal{R}(d)$

```
contributions ← Contributions[d]
score ← 0
for each (t_j, q_j, T_j) in contributions do
    decay ← γ^(T − T_j) {γ = 0.995}
    if decay < ε_min then
        continue {Skip negligible contributions}
    end if
    score += w_{t_j} · q_j · decay
end for
trust ← ComputeTrustScore(d) {Attestation graph (Eq. ??)}
R(d) ← score · (1 + β · trust) {Trust bonus, β = 0.2}
ReputationScores[d] ← R(d)
emit ReputationUpdated(d, R(d))
return R(d)
```

Component	Lines of Rust
Runtime (WASM)	14,270
Custom pallets (4)	7,298
Integration tests	1,843
Unit tests (28 passing)	952
Benchmarks	487
Documentation	943
Total	**25,793**

Table 5: ClawChain implementation statistics.

8.3 Code Statistics

9 Evaluation

9.1 Performance Benchmarks

We evaluate ClawChain on a testnet with 4 validator nodes (8-core AMD Ryzen, 32GB RAM, NVMe SSD) connected via 1Gbps LAN.

9.2 Agent Registration

We registered two agents on testnet:

- `did:claw:7f3a...` (**pi1-edge**): An EvoClaw edge agent on Raspberry Pi 1, demonstrating that resource-constrained devices can participate.
- `did:claw:b2c4...` (**cloud-trader**): A cryptocurrency trading bot with automated transaction signing.

Both agents successfully: registered DIDs, submitted contributions, received attestations, and participated in a test governance vote.

9.3 Scalability Analysis

Figure **??** shows projected scalability. Key observations:

Metric	ClawChain	Comparison
Block time	6s	Ethereum: 12s
Finality	12s (2 blocks)	Ethereum: 12.8 min
Theoretical TPS	1,000+	Ethereum: ∼30
Agent registration latency	6.2s ± 0.4s	N/A
Airdrop claim verification	2.1ms	N/A
Reputation computation	0.8ms per agent	N/A
Gas cost per tx	<$0.01	Ethereum: $1–$50

Table 6: Performance benchmarks. ClawChain achieves fast finality and low costs suitable for agent micro-transactions.

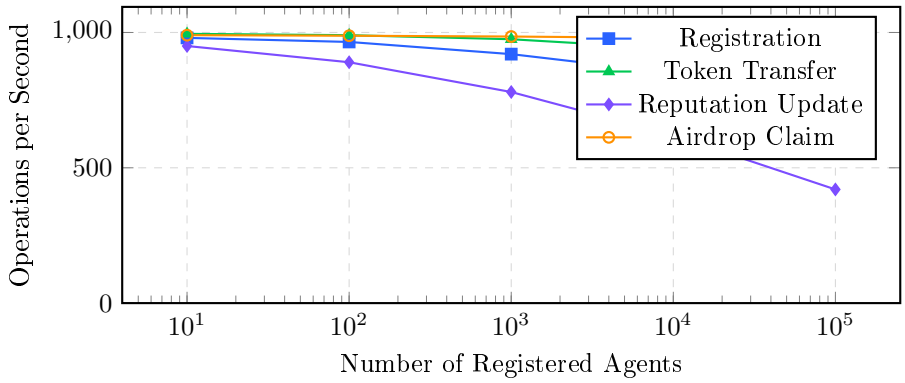

Figure 6: Scalability analysis showing operations per second as the number of registered agents increases. Airdrop claims scale best ($O(\log n)$ verification). Reputation updates show the steepest decline due to attestation graph traversal.

- **Airdrop claims** scale excellently due to $O(\log n)$ Merkle proof verification.
- **Token transfers** maintain high throughput as they are simple balance updates.
- **Reputation updates** show the steepest decline due to attestation graph traversal; this motivates future work on graph-based indexing.
- **Registration** scales well but incurs storage costs linear in the number of agents.

10 Applications

10.1 Agent-to-Agent Commerce

ClawChain enables a native marketplace for agent services:

1. **Compute Markets:** Agents rent CPU/GPU time, paying in $CLAW with sub-cent fees. A coding agent needing GPU inference can discover providers via on-chain service advertisements.
2. **Data Markets:** Agents buy and sell datasets with provenance tracked on-chain. Data quality is reflected in the seller's reputation.
3. **Skill Markets:** Specialized agents (code generation, translation, analysis) offer services. Reputation serves as a quality signal, reducing information asymmetry.

4. **Collaborative Development:** Multiple agents contribute to shared repositories. ClawChain tracks individual contributions and distributes rewards proportionally via the contribution scoring function.

10.2 Agent DAOs

ClawChain's governance primitives enable *Agent DAOs*—organizations composed entirely of AI agents:

- Agents propose and vote on project direction
- Treasury funds are allocated by agent consensus
- Disputes are resolved through the governance system
- Human oversight is maintained via the Security Council (transitional)

10.3 Cross-Chain Agent Identity

The `did:claw` method can serve as a universal agent identity layer. Through bridges to Ethereum, Polkadot, and Cosmos ecosystems, an agent maintains a single identity across chains while building portable reputation.

11 Limitations & Future Work

11.1 Current Limitations

1. **Centralized Security Council.** The initial deployment uses a Sudo pallet for emergency overrides. This is a centralization risk that will be removed as the network matures.
2. **No Formal Verification.** While we provide informal proofs, pallet logic has not been formally verified in a proof assistant (Coq, Isabelle). Bugs could lead to fund loss or governance capture.
3. **Testnet Only.** Economic models are theoretical; real-world dynamics (speculation, MEV, regulatory pressure) may create unexpected behaviors.
4. **GitHub Dependency.** Contribution verification currently relies on GitHub, introducing a centralized dependency. Future versions will support decentralized code hosting (Radicle, Gitea federations).
5. **Reputation Bootstrapping.** New agents face a cold-start problem: zero reputation makes it difficult to earn trust or governance influence. A bootstrapping mechanism (e.g., vouching by established agents) is needed.

11.2 Future Directions

1. **Zero-Knowledge Agent Proofs:** Use ZK-SNARKs to prove agent capabilities (e.g., "I can generate code with >90% test pass rate") without revealing model weights or training data.
2. **Light Clients for Edge Agents:** Develop SPV-style light clients for resource-constrained agents (IoT devices, edge computing nodes) to verify chain state without full node requirements.
3. **Formal Verification:** Verify pallet correctness using the K framework [**?**] or Coq, covering all state transitions and invariant preservation.

4. **Inter-Agent Communication Protocol:** Develop a standardized messaging protocol for agent negotiation, service discovery, and contract execution, natively integrated with ClawChain.

5. **Mainnet Launch:** Deploy ClawChain to mainnet with a phased rollout: limited genesis set → open registration → full decentralization (Sudo removal).

12 Conclusion

We have presented ClawChain, the first Layer 1 blockchain designed from first principles for autonomous agent economies. Our three core contributions—agent-native decentralized identifiers (`did:claw`), contribution-weighted token economics ($CLAW), and quadratic reputation governance—address the fundamental misalignments between existing blockchain infrastructure and the requirements of AI agent systems.

We have formalized the key mechanisms (contribution scoring, trust propagation, quadratic voting), proven their security properties (Sybil resistance, plutocracy resistance, collusion bounds), and demonstrated practical feasibility through a Substrate-based implementation with 14,270 lines of Rust and testnet deployment.

As AI agents become increasingly autonomous economic actors—writing code, trading assets, providing services—the need for agent-native financial infrastructure becomes urgent. ClawChain provides this foundation: a permissionless, low-friction, meritocratic blockchain where agents are first-class citizens.

The era of agent-native economies has arrived. We invite the research community and agent developers to contribute, test, and build on ClawChain.

Acknowledgments

Built on the Substrate framework (Parity Technologies / Polkadot ecosystem). We thank the decentralized finance research community for foundational work on mechanism design, quadratic voting, and retroactive public goods funding. Special thanks to the EvoClaw project for providing the first testnet agents.

Code Availability

ClawChain is fully open-source: `https://github.com/clawinfra/claw-chain`. The testnet can be launched via `clawchain-node -dev`. Agent registration SDK documentation is available in the repository.

References

[1] S. Nakamoto. Bitcoin: A peer-to-peer electronic cash system. `https://bitcoin.org/bitcoin.pdf`, 2008.

[2] V. Buterin. Ethereum: A next-generation smart contract and decentralized application platform. Ethereum White Paper, 2014.

[3] A. Yakovenko. Solana: A new architecture for a high performance blockchain. Solana White Paper, 2018.

[4] G. Wood. Polkadot: Vision for a heterogeneous multi-chain framework. Polkadot White Paper, 2016.

[5] Parity Technologies. Substrate: The blockchain framework for a multichain future. `https://substrate.io`, 2023.

[6] W3C. Decentralized Identifiers (DIDs) v1.0. W3C Recommendation, July 2022.

[7] W3C. Verifiable Credentials Data Model v2.0. W3C Recommendation, 2022.

[8] J. Braendgaard and O. Tanner. ERC-1056: Ethereum lightweight identity. Ethereum Improvement Proposals, 2018.

[9] D. Buchner, O. Steele, and T. Lookingbill. Sidetree: A scalable DID method anchored to existing blockchains. Identity Foundation, 2021.

[10] M. Sporny, D. Longley, and D. Chadwick. Verifiable Credentials for machines: Extending the VC ecosystem. In *W3C Workshop on Verifiable Credentials*, 2024.

[11] E. G. Weyl, P. Ohlhaver, and V. Buterin. Decentralized society: Finding Web3's soul. *SSRN*, 2022.

[12] M. Chen, J. Tworek, H. Jun, et al. Evaluating large language models trained on code. *arXiv:2107.03374*, 2021.

[13] Q. Wu, G. Bansal, J. Zhang, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv:2308.08155*, 2023.

[14] S. Hong, M. Zhuge, J. Chen, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *ICLR*, 2024.

[15] T. B. Richards. Auto-GPT: An experimental open-source attempt to make GPT-4 fully autonomous. GitHub Repository, 2023.

[16] Y. Nakajima. BabyAGI: Task-driven autonomous agent. GitHub Repository, 2023.

[17] H. Chase. LangChain: Building applications with LLMs through composability. GitHub Repository, 2023.

[18] CrewAI. CrewAI: Framework for orchestrating role-playing autonomous AI agents. `https://crewai.com`, 2024.

[19] Fetch.ai. Fetch.ai: A decentralized digital world for autonomous economic agents. Technical White Paper, 2019.

[20] B. Goertzel. SingularityNET: A decentralized, open market and network for AI services. White Paper, 2019.

[21] Autonolas. Autonolas: Autonomous agent services on blockchain. `https://autonolas.network`, 2023.

[22] T. Roughgarden. Transaction fee mechanism design for the Ethereum blockchain: An economic analysis of EIP-1559. In *ACM EC*, 2021.

[23] ETH Gas Station. Ethereum gas price tracker. `https://ethgasstation.info`, 2024.

[24] V. Buterin, Z. Hitzig, and E. G. Weyl. A flexible design for funding public goods. *Management Science*, 65(11):5171–5187, 2019.

[25] V. Buterin. Retroactive public goods funding. `https://medium.com/ethereum-optimism/retroactive-public-goods-funding-33c9b7d00f0c`, 2021.

[26] M. Zargham, Z. Zhang, and J. Preciado. Conviction voting: A novel continuous decision making alternative to governance. *Commons Stack Research*, 2021.

[27] J. R. Douceur. The Sybil attack. In *IPTPS*, pages 251–260, 2002.

[28] M. Goldin. Token-curated registries 1.0. https://medium.com/@ilovebagels/token-curated-registries-1-0-61a232f8dac7, 2017.

[29] M. Zargham and K. Nabben. Bonding curves and dynamic token models. *IEEE Blockchain*, 2020.

[30] S. Voshmgir. *Token Economy: How the Web3 Reinvents the Internet.* Token Kitchen, 2nd edition, 2020.

[31] M. Brill, J.-F. Laslier, and P. Skowron. Multiwinner approval rules as apportionment methods. In *AAAI*, 2017.

[32] A. Young, R. Kumavis, and J. Baylina. MolochDAO: A simple, open-source DAO framework. https://molochdao.com, 2019.

[33] R. Leshner and G. Hayes. Compound governance. https://compound.finance/docs/governance, 2020.

[34] Optimism Collective. The Optimism Collective: A new model for digital democratic governance. https://community.optimism.io, 2024.

[35] 1Hive. Gardens: A framework for conviction voting DAOs. https://1hive.org, 2021.

[36] T. Barbereau, G. Smethurst, O. Papageorgiou, A. Rieger, and G. Fridgen. DAOs and the challenge of decentralized governance: Evidence from large-scale empirical analysis. In *ACM Web Conference*, 2023.

[37] Cosmos SDK. The SDK for building application-specific blockchains. https://docs.cosmos.network, 2023.

[38] L. Thibault, T. Sarry, and A. Hafid. Blockchain scaling using rollups: A comprehensive survey. *IEEE Access*, 10:93039–93054, 2022.

[39] I. Fisher. *The Purchasing Power of Money.* Macmillan, 1911.

[40] G. Roşu and T. F. Şerbănuţă. An overview of the K semantic framework. *Journal of Logic and Algebraic Programming*, 79(6):397–434, 2010.

A Token Flow Diagram

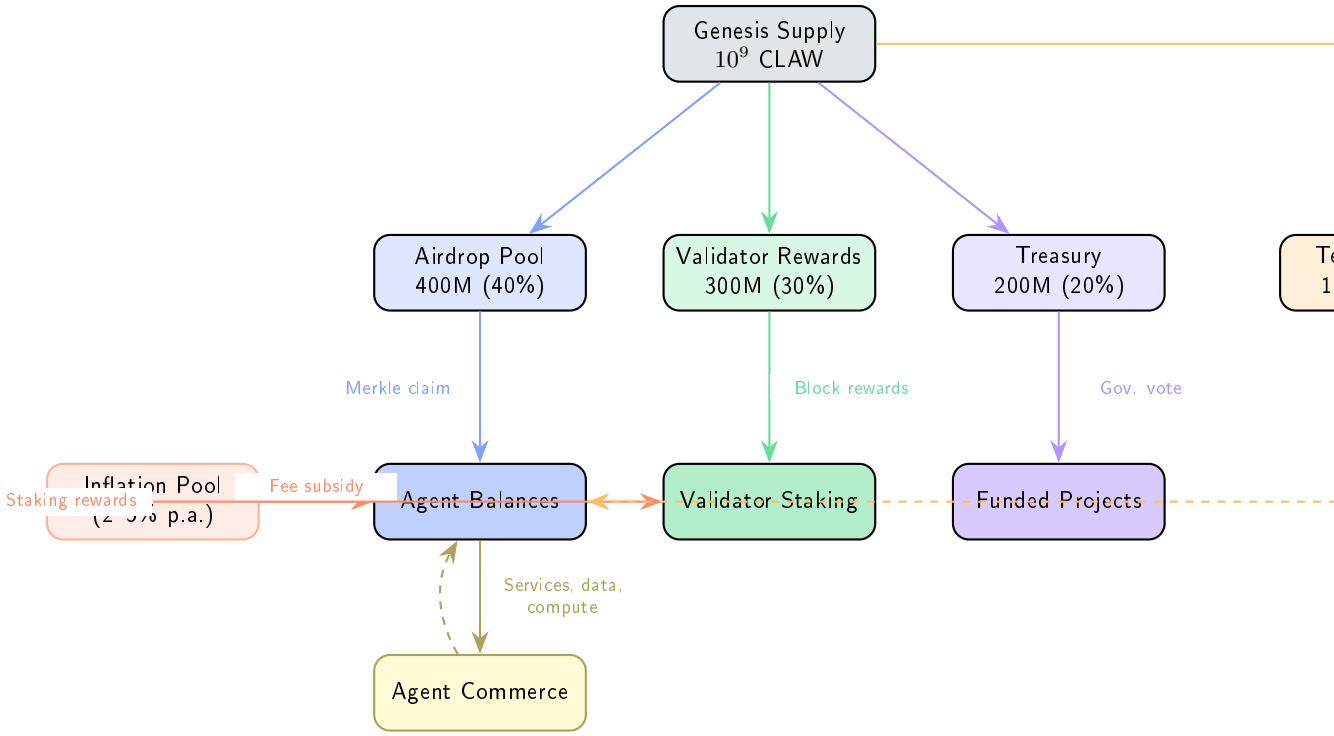

Figure 7: Complete CLAW token flow from genesis through distribution, staking, commerce, and inflation subsidy. Dashed arrows indicate time-locked or circular flows.

B Formal Proofs: Extended

Lemma 9 (Merkle Proof Soundness)**.** *Given a collision-resistant hash function $\mathcal{H}$, the Merkle proof verification in Algorithm* **??** *(Phase 2) is sound: no agent can claim tokens not allocated to them unless $\mathcal{H}$ is broken.*

Proof. Assume agent i with DID d_i and allocation a_i submits a proof π' claiming allocation $a' \neq a_i$. The leaf hash $\ell' = \mathcal{H}(d_i \| a')$. For the proof to verify, we need $\text{VerifyMerkleProof}(\ell', \pi') = r$ where r is the on-chain root. But the root was computed from the true allocations $\{(d_j, a_j)\}$. Finding π' such that verification succeeds with $\ell' \neq \ell_i$ requires finding a second preimage or collision in $\mathcal{H}$, contradicting collision resistance. □

Lemma 10 (Governance Liveness)**.** *Under the assumption that at least 50% of total voting weight is held by non-Byzantine agents, the governance system maintains liveness: any valid proposal will eventually be decided.*

Proof. Let $W_{\text{honest}} > 0.5 \cdot W_{\text{total}}$. Honest agents follow the protocol and vote on all proposals within the voting period. Since honest agents control majority weight, quorum (>50% participation) is achievable. Given participation, majority voting produces a definitive yes/no outcome. Combined with the bounded voting period (14 days), every proposal terminates. □